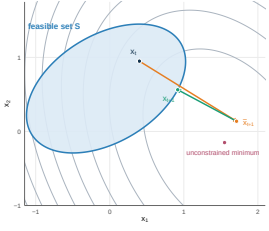
Optimization in Machine Learning

Constrained Optimization
Primal Gradient Descent Methods



Learning goals

- Why unconstrained GD fails under constraints
- Projected gradient descent and projected gradients
- Equality, box, and general convex constraints
- Convergence facts and ML examples

WHY CONSTRAINTS BREAK GD

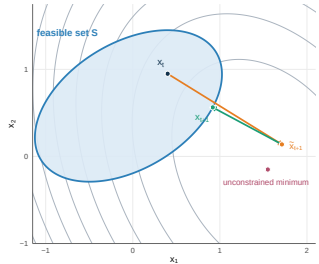
Problem

$$\min_{\boldsymbol{x}\in\mathcal{S}} f(\boldsymbol{x}), \text{ with } \mathcal{S} \subsetneq \mathbb{R}^d$$

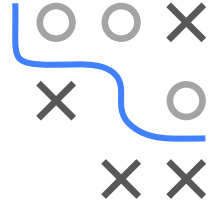
Vanilla GD step

$$\mathbf{z}^{[t+1]} = \boldsymbol{x}^{[t]} - \alpha_t \nabla f(\boldsymbol{x}^{[t]}).$$

- $\boldsymbol{x}^{[t]}$ can be feasible while $\mathbf{z}^{[t+1]} \notin \mathcal{S}$.
- Primal methods modify or repair the step so every iterate stays inside $\mathcal{S}$.
- Roadmap: feasible updates first, relaxation next, then hybrid methods.



PROJECTED GRADIENT DESCENT



Algorithm Projected Gradient Descent

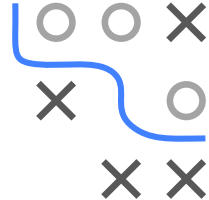
```
Choose a feasible start x^[0] ∈ S
while stopping criterion not met do
    z^[t+1] ← x^[t] − α_t ∇f(x^[t])
    x^[t+1] ← Π_S(z^[t+1])
end while
```

- The projection operator is

$$\Pi_{\mathcal{S}}(\mathbf{z}) := \underset{\boldsymbol{x}\in\mathcal{S}}{\arg\min}\, \|\boldsymbol{x} - \mathbf{z}\|_2^2.$$

- If the tentative point is already feasible, the projection does nothing.
- For a closed convex set $\mathcal{S}$, the Euclidean projection is unique.

PROJECTED GRADIENTS AND OPTIMALITY



- A constrained optimum $\boldsymbol{x}^*$ is no longer characterized by $\nabla f(\boldsymbol{x}^*) = \mathbf{0}$.
- Instead, for every $\alpha > 0$,

$$\boldsymbol{x}^* = \Pi_{\mathcal{S}}\big(\boldsymbol{x}^* - \alpha \nabla f(\boldsymbol{x}^*)\big)$$

  is the fixed-point condition for first-order optimality over a convex set.
- Equivalently, the negative gradient must lie in the normal cone:

$$-\nabla f(\boldsymbol{x}^*) \in N_{\mathcal{S}}(\boldsymbol{x}^*).$$

- Define the **projected gradient mapping**

$$G_\alpha(\boldsymbol{x}) := \frac{1}{\alpha}\big(\boldsymbol{x} - \Pi_{\mathcal{S}}(\boldsymbol{x} - \alpha \nabla f(\boldsymbol{x}))\big).$$

- $G_\alpha(\boldsymbol{x}) = \mathbf{0}$ is the constrained analogue of $\nabla f(\boldsymbol{x}) = \mathbf{0}$.
- If $\mathcal{S} = \mathbb{R}^d$, then $\Pi_{\mathcal{S}}$ is the identity and $G_\alpha(\boldsymbol{x}) = \nabla f(\boldsymbol{x})$.
- Practical stopping rules therefore monitor $\|G_\alpha(\boldsymbol{x}^{[t]})\|_2$ or the decrease in $f(\boldsymbol{x}^{[t]})$.

EQUALITY CONSTRAINTS: FEASIBLE DIRECTIONS

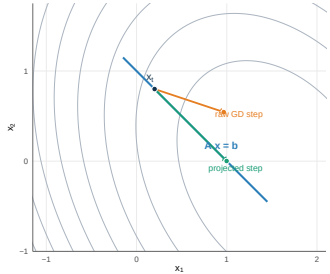
Problem

$$\min f(\boldsymbol{x}) \qquad \text{s.t.} \qquad \mathbf{A}\boldsymbol{x} = \mathbf{b}.$$

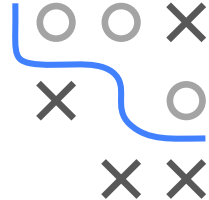
Feasible directions

$$\mathbf{A}\mathbf{d}^{[t]} = \mathbf{0}.$$

- The raw negative gradient usually violates feasibility.
- Hence we descend only along the tangent / null-space directions.



EQUALITY CONSTRAINTS: ALGEBRAIC FORM



- Assume the rows of $\mathbf{A} \in \mathbb{R}^{m \times d}$ are linearly independent.
- The orthogonal projector onto the feasible directions is

$$\Pi_{\mathrm{null}(\mathbf{A})} = \boldsymbol{I} - \mathbf{A}^\top (\mathbf{A}\mathbf{A}^\top)^{-1}\mathbf{A}.$$

- Therefore, the projected-GD update becomes

$$\boldsymbol{x}^{[t+1]} = \boldsymbol{x}^{[t]} - \alpha_t \Pi_{\mathrm{null}(\mathbf{A})} \nabla f(\boldsymbol{x}^{[t]}).$$

- Equivalent viewpoint: parameterize all feasible points as

$$\boldsymbol{x} = \boldsymbol{x}_0 + \mathbf{N}\mathbf{z}, \qquad \mathbf{A}\boldsymbol{x}_0 = \mathbf{b}, \qquad \mathbf{A}\mathbf{N} = \mathbf{0},$$

  and run ordinary GD on the reduced variable $\mathbf{z}$.
- Equality constraints are therefore often simpler than general inequalities: feasibility is linear, and the feasible directions have an explicit subspace structure.

TOY EXAMPLE: BOX CONSTRAINTS

Toy problem

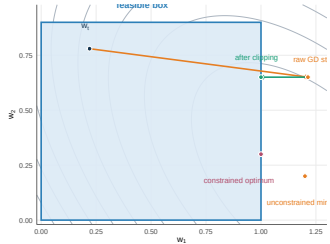
$$\min_{\boldsymbol{w}\in\mathcal{S}} F(\boldsymbol{w}) := (w_1 - 1.2)^2 + (w_1 - 1.2)(w_2 - 0.2) + (w_2 - 0.2)^2$$

$$\mathcal{S} = [0, 1] \times [0, 0.9].$$

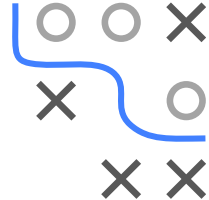
Projection

$$[\Pi_{\mathcal{S}}(\mathbf{z})]_1 = \min\{1, \max\{0, z_1\}\},$$
$$[\Pi_{\mathcal{S}}(\mathbf{z})]_2 = \min\{0.9, \max\{0, z_2\}\}.$$

- The unconstrained minimizer is $(1.2, 0.2)$, so it lies just outside the box.
- A projected-GD step repairs only the violated coordinates by clipping.
- We reuse this same box-constrained quadratic in the next sections.



GENERAL CONVEX CONSTRAINTS

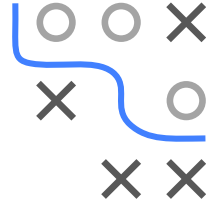


- For any closed convex feasible set $\mathcal{S}$, projected GD still takes the form

$$\boldsymbol{x}^{[t+1]} = \arg\min_{\boldsymbol{x}\in\mathcal{S}} \left\| \boldsymbol{x} - \left(\boldsymbol{x}^{[t]} - \alpha_t \nabla f(\boldsymbol{x}^{[t]})\right) \right\|_2^2 .$$

- The key question is therefore not only how hard it is to differentiate f, but also how hard it is to compute $\Pi_{\mathcal{S}}$.
- Examples with cheap projectors:
  - affine sets and linear equalities
  - box constraints and nonnegativity constraints
  - simplices and some norm balls
- Examples with more expensive projectors:
  - general polyhedra $\{\boldsymbol{x} \mid \mathbf{A}\boldsymbol{x} \leq \mathbf{b}\}$
  - complicated nonlinear convex constraint sets
- This dependence on constraint structure is a recurring theme throughout constrained optimization.

ML EXAMPLES FOR PRIMAL PROJECTED METHODS



- **Nonnegative least squares**

$$\min_{\boldsymbol{w}} \|\mathbf{X}\boldsymbol{w} - \boldsymbol{y}\|_2^2 \qquad \text{s.t.} \qquad \boldsymbol{w} \geq \mathbf{0}$$

  uses the box/nonnegativity projector.

- **Probability vectors** in multinomial models satisfy

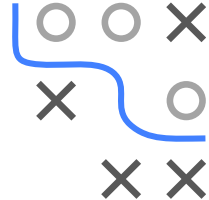
$$\boldsymbol{\theta} \geq \mathbf{0}, \qquad \mathbf{1}^\top \boldsymbol{\theta} = 1,$$

  so every step must remain on the simplex.

- **Bounded parameter models** and **budget constraints** naturally yield lower and upper bounds on variables.

- In all of these cases, the “primal” viewpoint is intuitive:
  - work directly with the original coefficients
  - keep iterates feasible
  - exploit the geometry of the feasible set

STEP SIZES AND CONVERGENCE

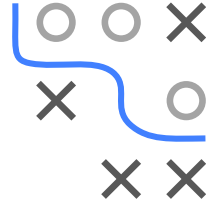


- Assume f is convex with L-Lipschitz continuous gradient and $\mathcal{S}$ is closed and convex.
- With a constant step size $0 < \alpha_t \leq 1/L$ or with backtracking line search, projected GD keeps all iterates feasible and converges to a minimizer.
- Typical rates mirror unconstrained GD:
  - convex case: $f(\boldsymbol{x}^{[t]}) - f(\boldsymbol{x}^*) = \mathcal{O}(1/t)$
  - μ-strongly convex case: linear convergence
- The computational cost per iteration is

$$\text{one gradient evaluation} + \text{one projection}.$$

- Hence projected GD is attractive exactly when the projector is simple enough that this extra step is cheap.

TAKE-AWAY



1. Start from a feasible point.
2. Take a standard gradient step in the original variables.
3. Project back onto the feasible set.
4. Exploit structure aggressively:
   - equalities $\rightarrow$ null-space / tangent-space projection
   - box constraints $\rightarrow$ clipping
   - simple convex sets $\rightarrow$ specialized projectors

Projected GD is the default primal baseline when projection onto $\mathcal{S}$ is cheap.